

Analog & Digital Conversion

Making Sense of the Real World



12-bit ADC Core Math

The RP2040 ADC converts an analog input into its 12-bit digital representation.

The upper reference is V_{REF} (typically 3.3V or 5V) and lower reference is Ground (typically 0V).

$$N_{ADC} = (2^{12} - 1) \times \frac{V_{in}}{V_{REF}}$$

Max: $N_{ADC} = 4095$ when $V_{in} \geq V_{REF}$

Min: $N_{ADC} = 0$ when $V_{in} \leq 0V$

Quantisation error is the difference between the actual analog voltage and the nearest digital value the ADC can represent, typically up to ± 0.5 LSB.

Practice Calculation

Q1: What is the digital output when a potentiometer is set to 1.23V (Assuming)? $V_{REF} = 3.3V$

$$N_{ADC} = 4095 * (1.23V / 3.3V)$$

$$N_{ADC} \approx 1526$$

Q2: What is the LSB resolution?

$$3.3V / 4096 = 0.000805V = 0.8mV$$

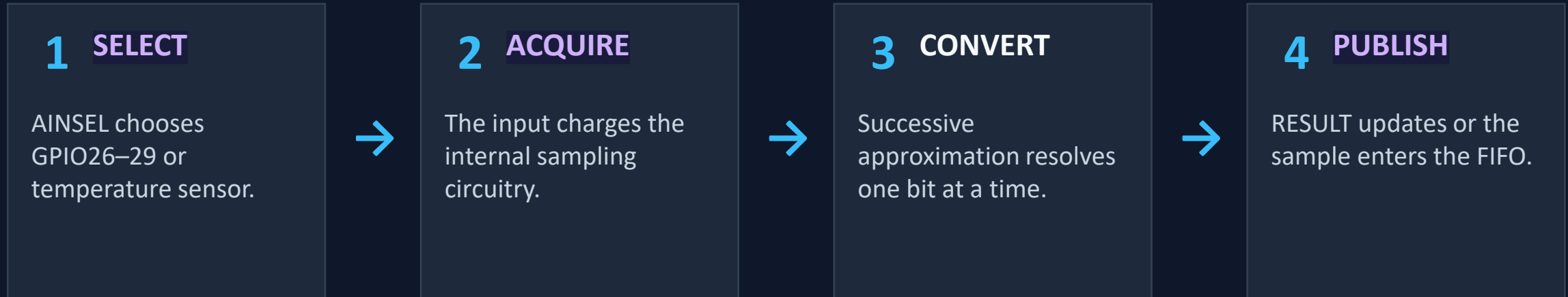
Q3: What is the maximum Quantisation error?

$$\pm (\frac{1}{2} * \text{LSB}) \approx \pm 0.403 \text{ mV}$$



The ADC acquires voltage before conversion

RP2040 uses one 12-bit SAR converter behind a five-input multiplexer.



Maximum-rate timing

48 MHz ADC clock
96 cycles per conversion

One complete result

2 μ s

Takeaway

The signal must settle before bit decisions can be trusted.



Twelve output bits do not guarantee **twelve accurate bits**

Resolution describes code spacing; accuracy describes closeness to the true voltage.

Ideal 12-bit scale

4096

possible codes
0 to 4095

At 3.3 V: 1 LSB $\approx$ 0.806 mV

Repeat the same DC measurement: spread reveals noise; average reveals bias.

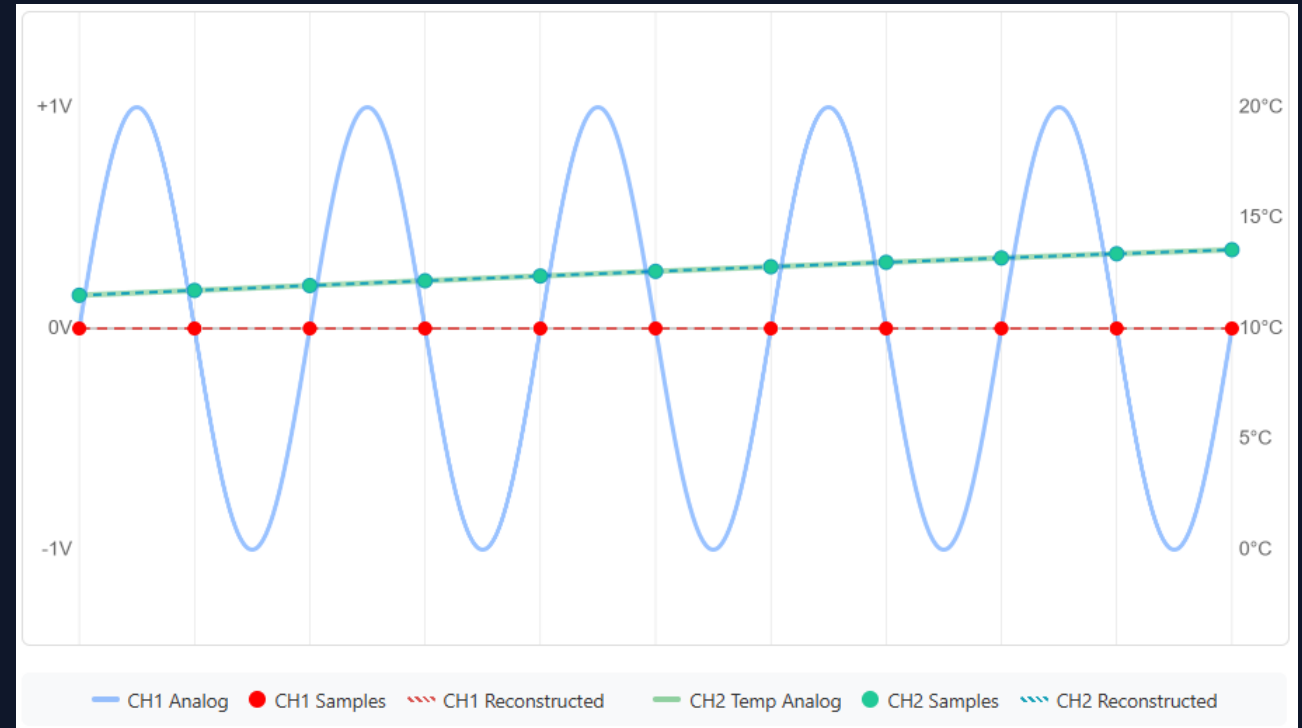
Why measured accuracy is lower

- **Reference:** noise or drift changes every code.
- **Offset + gain:** the transfer curve shifts or tilts.
- **Nonlinearity:** code steps are not perfectly uniform.
- **Noise:** repeated readings spread across codes.
- **Effective resolution:** RP2040 is specified at 8.7 ENOB.



The Zero-Crossing Trap (Nyquist Limit)

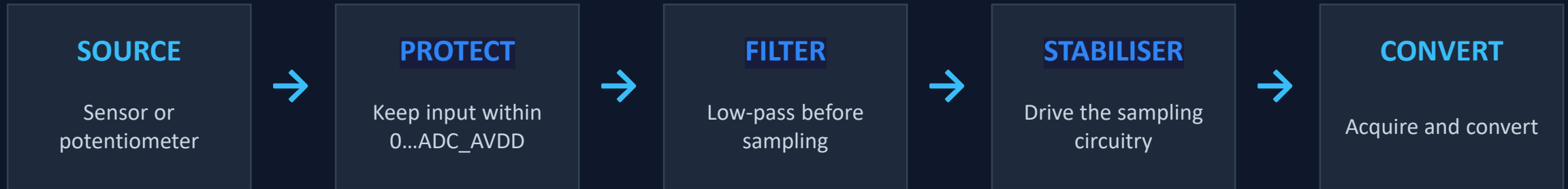
- Nyquist states we must sample at $f_s > 2f_{\max}$
- **Critical Sampling** (exactly 2x) is dangerous in embedded systems.
- If sampling perfectly aligns with the signal's zero-crossings, the ADC reads all zeros.
- The reconstructed signal appears as a flat DC line (0V), losing the original waveform entirely.
- **Solution:** Always oversample (e.g., 2.5x to 3x) and use hardware timers, never software delays, to trigger the ADC.





Good ADC software cannot rescue a **bad analog front end**

Condition the signal before it reaches GPIO26–29.



GPIO setup

Use `adc_gpio_init()`:

- **high impedance**
- **pulls disabled**
- **digital input disabled**

Low-pass filter

Limit analog bandwidth before sampling. Frequencies that alias cannot be removed later in software.

Reference + ground

ADC codes are ratios to ADC_AVDD. Noise on the reference or return path appears as measurement noise.



Most “bad ADC” readings begin before the conversion starts

Diagnose the symptom, then check the electrical cause before rewriting firmware.

SYMPTOM	LIKELY CAUSE	FIX
Codes wander at rest	Floating input or noisy reference	Add a defined source; improve grounding and decoupling.
First sample is wrong	Source did not settle after a channel change	Allow settling time; buffer high-impedance sources.
Waveform has a false frequency	Input bandwidth exceeds the sample plan	Add an analog low-pass filter; raise sample rate.
Readings clip at an endpoint	Input is outside the valid ADC range	Scale and protect the input; never exceed <code>ADC_AVDD</code> .
Channels influence each other	Shared converter plus incomplete settling	Slow the scan, discard a settling sample, or buffer.



Firmware Calibration



The Problem

Silicon is imperfect. Manufacturing tolerances introduce Offset Errors (curve shifted up/down) and Gain Errors (slope too steep/shallow).



Two-Point Method

Calculate true slope and intercept using known low and high voltages.

$$\text{Gain} = \frac{V_{hi} - V_{lo}}{ADC_{hi} - ADC_{lo}}$$

$$\text{Offset} = V_{lo} - (\text{Gain} \times ADC_{lo})$$



RTOS Implementation

Execute in a task, not the [ISR](#). Use fixed-point math to avoid slow floating-point operations.

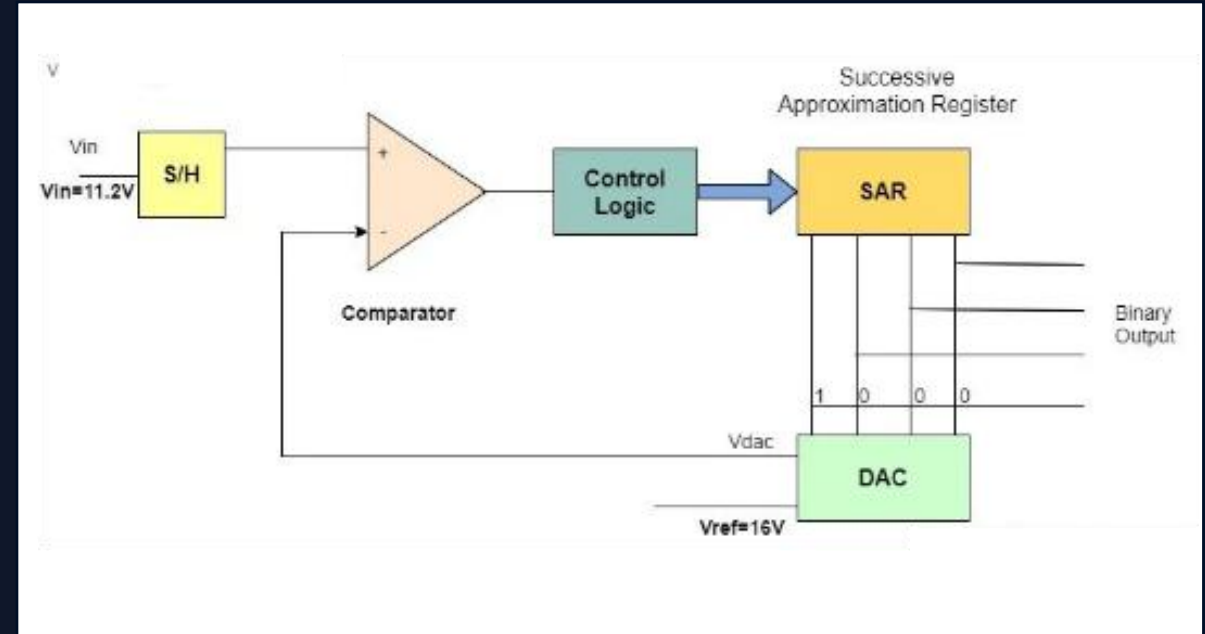
$$V_{\text{actual}} = (\text{ADC} \times \text{Gain}) + \text{Offset}$$



The RP2040 ADC samples five sources at up to **500 kSPS**

- 12-bit conversion: 0–4095 output codes.
- Throughput: up to 500 **kSPS**.
- External inputs: GPIO26–29 (channels 0–3).
- Internal input: temperature sensor (channel 4).
- Buffering: 8-entry FIFO.
- Streaming: DMA supports CPU-independent transfer.

Input mux → 12-bit converter → **RESULT** or **FIFO**



Five selectable sources share one converter; read samples directly or stream them through the FIFO to DMA.



ADC_CS controls power, channel, and conversion mode

Use the control/status register to enable the ADC, select an input, start conversions, and check readiness.

Bit(s)	Field	Purpose
0	EN	Power on the ADC and enable its clock.
1	TS_EN	Power on the internal temperature sensor.
2	START_ONCE	Start one conversion; the bit self-clears.
3	START_MANY	Run conversions continuously.
8	READY	1 when ready for a new conversion; 0 while busy.
14:12	AINSEL	Select GPIO26–29 (0–3) or temperature sensor (4).

Typical sequence

- 1 Enable EN
- 2 Select AINSEL
- 3 Set START_ONCE or START_MANY
- 4 Check READY or read FIFO

START_ONCE self-clears; START_MANY keeps the converter running.



RESULT is for polling; FIFO is for continuous capture

Single polling mode

RESULT register

- Holds the most recent 12-bit conversion result.
- Reading RESULT does not clear interrupt flags.
- Best matched to manual START_ONCE conversions.

Streaming mode

FCS + FIFO registers

- The ADC provides an 8-entry FIFO buffer.
- FCS controls enable, threshold, underflow, and overflow status.
- Reading FIFO removes the oldest 12-bit sample.
- FIFO decouples the CPU from continuous START_MANY acquisition.

**Use RESULT for occasional synchronous reads;
Use FIFO when samples arrive faster than the CPU should block.**

Conversion completion updates **RESULT** or **FIFO**



At 500 kSPS, a full-rate sample completes every 2 μ s.

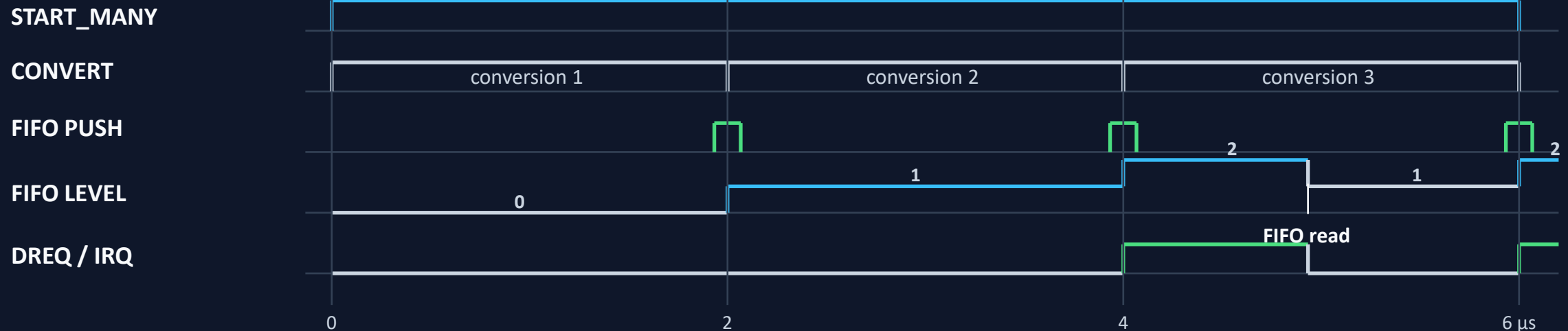
Single conversion — **START_ONCE**

not to scale



Continuous conversion — **START_MANY + FIFO**

8-entry FIFO · threshold example = 2

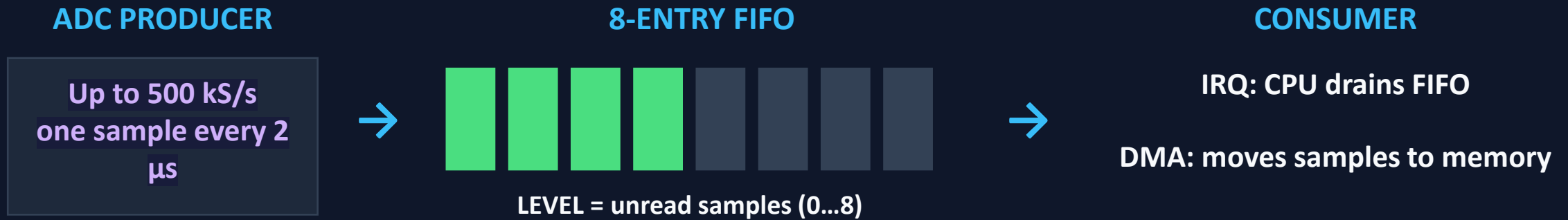


FIFO LEVEL counts unread samples (0–8); requests assert while LEVEL $\geq$ THRESH.



The 8-entry FIFO does not prevent **data loss**

Reliable capture requires the consumer to keep pace with the ADC.



Threshold choice

- **1:** lowest latency, highest IRQ rate.
- **4:** balanced batching and margin.
- **8:** maximum batching, no overflow margin.

Failure signals

- **OVER:** producer outran consumer.
- **UNDER:** software read an empty FIFO.
- **ERR:** conversion error can accompany data.

Design rule

Average drain rate must exceed the sample rate, and the buffer must absorb the longest processing stall.



Polling blocks the loop; FIFO interrupts keep acquisition moving

Polling — synchronous read

Simple control flow; the CPU waits for `adc_read()`.

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"

int main(void)
{
    stdio_init_all();
    adc_init();
    adc_gpio_init(26);
    adc_select_input(0);

    while (true)
    {
        uint16_t result = adc_read();
        float voltage = result * (3.3f / 4096.0f);
        printf("Raw: %u, V: %.2f\n", result, voltage);
        sleep_ms(100);
    }
}
```

Best for occasional samples and straightforward logic.

FIFO interrupt — continuous read

The ADC runs continuously; the ISR drains queued samples.

```
#include "hardware/adc.h"
#include "hardware/irq.h"

void adc_isr(void)
{
    while (!adc_fifo_is_empty())
    {
        uint16_t result = adc_fifo_get();
        // Process or buffer quickly
    }
}

int main(void)
{
    adc_init();
    adc_gpio_init(26);
    adc_select_input(0);
    adc_fifo_setup(true, false, 1, false, false);
    irq_set_exclusive_handler(ADC_IRQ_FIFO, adc_isr);
    adc_irq_set_enabled(true);
    irq_set_enabled(ADC_IRQ_FIFO, true);
    adc_run(true);

    while (true) tight_loop_contents();
}
```

Best for sustained capture without blocking the main loop.



Slow firmware turns ADC samples into **data loss**

Keep the sampling path deterministic and move expensive work elsewhere.

COMMON FAILURE

Long ISR

Formatting, floating point, or printf delays FIFO draining.

Threshold = 1 by habit

Interrupt overhead can dominate at high sample rates.

Read only one sample

Queued samples remain and the IRQ may immediately retrigger.

WHY IT HURTS

Software-delay timing

sleep_ms() adds jitter and does not guarantee cadence.

No software buffer

Temporary processing stalls become FIFO overflow.

Unsafe shared data

ISR and main code race on counters, indices, or sample arrays.

Use polling for occasional reads, IRQ for moderate streams, and DMA for sustained high-rate capture.



Debug ADC systems by testing one uncertainty **at a time**

A six-step lab turns vague “noisy ADC” reports into measurable evidence.

1 KNOWN DC

Apply a safe known voltage and predict the ideal code.

2 REPEAT

Capture 1,000 samples; calculate mean, min, max, and spread.

3 REFERENCE

Compare ADC_AVDD and ground behavior with a multimeter.

4 TIMING

Change sample rate and look for settling or aliasing changes.

5 STRESS

Slow the consumer deliberately and observe FIFO OVER.

6 COMPARE

Measure polling, IRQ, and DMA CPU cost and jitter.

Exit criterion: you can explain every unexpected code using electrical, timing, or software evidence.